JADE™ GLOBAL

LLM Advanced- P1

LLM Agents,
LangGraph WorkFlows,
Multi-Agent Systems,
Observability

©2025 Jade Global All rights reserved

Intermediate Recap

Everything you need from LLM Intermediate before going Advanced

01

Prompt Engineering

Role prompting, CoT, ReAct, chaining — craft precise, powerful instructions.

02

LLM APIs

temperature, max_tokens, streaming, retry logic — calling LLMs in code.

03

LangChain Basics

LCEL chains, memory types, agents, tool use — the app framework.

04

Embeddings

Text → vectors. Cosine similarity → semantic search. Models: ada-002, MiniLM.

05

Vector Databases

Chroma, Pinecone, FAISS. HNSW indexing. ANN search at scale.

06

RAG

Load → Chunk → Embed → Retrieve → Prompt → Answer. Ground LLMs in your data.

What We'll Cover Today

Your roadmap for LLM Advanced – Part 1

01 LLM Agents

LLMs that take actions, not just answer

- What makes an agent
- Tool use in practice
- Real-world agent examples
- Building your first agent

02 LangGraph Workflows

Visual, stateful LLM pipelines

- Nodes, edges & state
- Conditional branching
- Loops & human-in-the-loop
- Real workflow examples

03 Multi-Agent Systems

Teams of AI agents solving big problems

- Why multiple agents
- Supervisor & worker pattern
- Agent communication
- Real industry examples

04 Observability

Seeing inside your LLM app

- Tracing & logging
- LangSmith walkthrough
- Debugging failures
- Monitoring in production

MODULE 01

LLM Agents

LLMs that plan, decide, and take real-world actions

What is an LLM Agent?

An LLM that doesn't just answer — it acts

💡 *A regular LLM answers questions. An Agent uses an LLM as a brain to decide which tools to call, in what order, to complete a real-world task.*

Regular LLM

User: "Book me a flight to Delhi next Monday"

LLM: "Sure! Here's how you can book a flight...
1. Go to MakeMyTrip or Cleartrip
2. Enter your destination..."

❌ Just gives instructions. Takes no action.

LLM Agent

User: "Book me a flight to Delhi next Monday"

Agent thinks:
→ search_flights("Delhi", "Monday")
→ check_calendar("Monday")
→ book_flight(flight_id, user_card)
→ send_confirmation_email()

✅ Calls real tools. Task is done.

The Agent Loop

Think → Act → Observe → Repeat until done

User Goal → 🧠 Think (LLM Plans) → 🔧 Act (Call Tool) → 👁️ Observe (Tool Result) → ✅ Final Answer

↺ Loop repeats until task is complete or LLM decides it's done

Real Example: "What's the weather in Mumbai and should I carry an umbrella?"

Think	Act	Observe	Think
I need current Mumbai weather. I'll call the weather API.	→ weather_tool(city="Mumbai")	Result: 28°C, 85% humidity, rain likely after 3pm.	User asked about umbrella. Rain is likely. I can now answer.

Real-World Agent Applications

Where agents are being used right now

🎧 Customer Support

Customer says: "My order hasn't arrived"

```
→ lookup_order(customer_id)
→ track_shipment(order_id)
→ check_delay_reason()
→ send_apology_voucher()
```

Used by: Intercom, Freshdesk, Zendesk AI

💻 Software Dev

"Fix the failing unit tests in my repo"

```
→ read_test_file()
→ run_tests()
→ identify_failing_test()
→ edit_code()
→ run_tests() again
```

Used by: GitHub Copilot Workspace, Cursor, Devin

📈 Finance

"Summarise Q3 earnings for top 5 IT stocks"

```
→ search_web("Q3 earnings TCS")
→ fetch_url(report_url)
→ extract_numbers()
→ compare_yoy()
→ write_summary()
```

Used by: Bloomberg GPT, Morgan Stanley AI

Building Your First Agent

A customer support agent in ~20 lines of Python

What you need:

🧠 **LLM**

The brain — GPT-4o or Claude 3

🔧 **Tools**

Python functions the agent can call

💬 **Memory**

Stores conversation so far

📋 **System Prompt**

Defines agent role and limits

```
# Simple AI-like support agent (no external libraries)

def check_order(order_id):
    return f"Order {order_id} is being processed."

def send_discount(email):
    return f"Discount code sent to {email}!"

def support_agent(user_input):
    if "order" in user_input.lower():
        return check_order("5678")
    elif "discount" in user_input.lower():
        return send_discount("user@example.com")
    else:
        return "Sorry, I didn’t understand your request."

# Run the agent
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break
    response = support_agent(user_input)
    print("Agent:", response)
```

MODULE 02

LangGraph Workflows

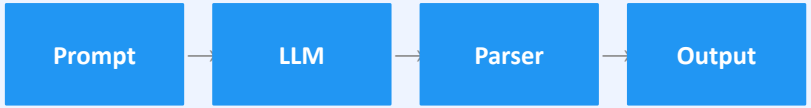
Stateful, visual pipelines for complex LLM logic

What is LangGraph?

When your LLM workflow needs branches, loops, and state

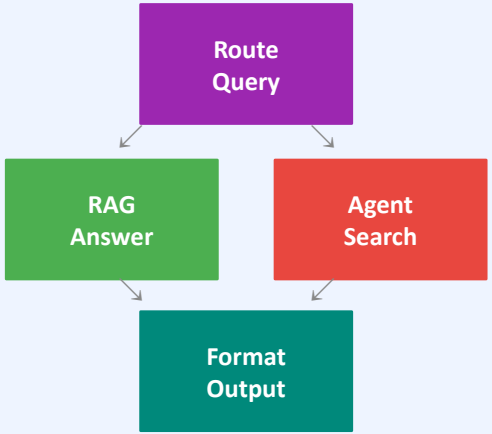
💡 *LangGraph lets you define LLM workflows as a graph — nodes are processing steps, edges decide what happens next. Think of it as a flowchart your LLM app actually runs.*

LangChain Chain (Linear — works for simple tasks)



✅ *Good for: Simple Q&A, summarization, one-shot tasks*

LangGraph Workflow (Branching — for complex tasks)



✅ *Good for: Multi-step tasks, decisions, human approval loops*

LangGraph: Nodes, Edges & State

The three building blocks of every LangGraph app

State

A shared Python dict that flows through every node. Each node can read and update it.

```
state = {
  "messages": [...],
  "user_intent": "booking",
  "flights_found": [...]
}
```

Nodes

Python functions that process the state and return an updated version. Each node = one step.

```
def search_flights(state):
  results = flight_api(
    state["destination"])
  return {"flights": results}
```

Edges

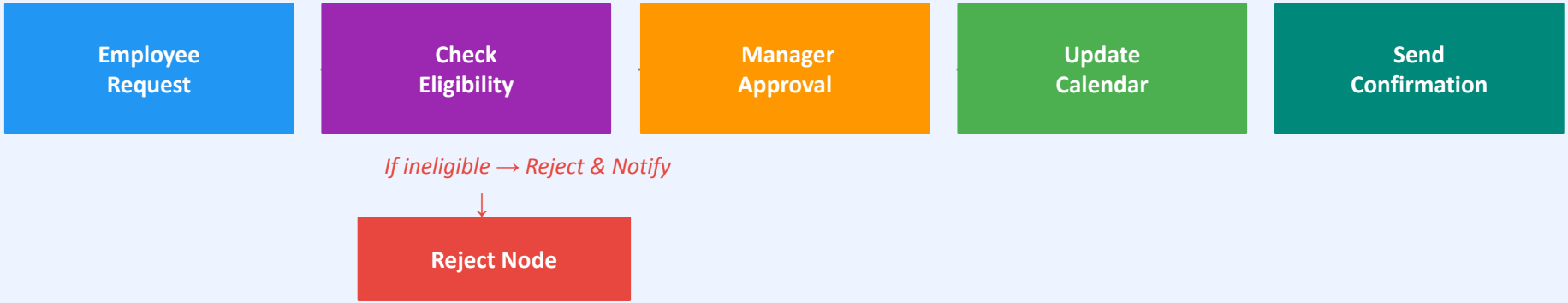
Connections between nodes. Can be fixed (always goes to B after A) or conditional (depends on state).

```
graph.add_conditional_edges(
  "classify",
  lambda s: s["intent"],
  {"book": "book_node",
   "cancel": "cancel_node"})
```

LangGraph Real Example

An HR leave approval workflow with human-in-the-loop

🏢 *Real use case: Jade Global HR bot. Employee asks for leave → AI checks eligibility → Manager approves → Calendar updated. LangGraph manages the whole flow.*



👤 Human-in-the-Loop:

The Manager Approval node pauses the graph and waits for a real human to click Approve or Deny. This is LangGraph's interrupt() feature — perfect for anything requiring human judgment.

MODULE 03

Multi-Agent Systems

Teams of AI agents tackling big problems together

Why Multiple Agents?

One agent is a tool. Multiple agents are a team.

💡 *Just like you wouldn't ask one person to be your accountant, lawyer, and coder — you give each AI agent a focused role. Better results, less confusion, easier to debug.*

❌ Single Agent for Everything

- Context window overflows on long tasks
- Hard to specialise — "do it all" = mediocre at everything
- Single point of failure — one mistake fails the whole task
- Hard to debug — which step went wrong?
- Slow — tasks that can be parallelised run sequentially

✅ Multi-Agent Team

- Each agent has a focused role — specialist beats generalist
- Tasks run in parallel — 5x faster on complex workflows
- Supervisor catches and routes errors between agents
- Easy to test, swap, or upgrade one agent independently
- Scales to arbitrarily complex real-world workflows

Multi-Agent Patterns

The three most common ways to organise AI agent teams

👔 Supervisor + Workers

One supervisor agent reads the goal, assigns sub-tasks to specialist workers, and assembles the final result.

```
Supervisor → assigns to:
• Researcher Agent
• Writer Agent
• Fact-checker Agent
```

📌 *Use for: Content generation, research reports*

🏭 Pipeline (Assembly Line)

Agents pass work along a fixed sequence. Output of one becomes input of the next.

```
Scraper Agent
  → Summariser Agent
    → Translator Agent
      → Email Agent
```

📌 *Use for: Data pipelines, document processing*

🌐 Peer Network

Agents communicate freely — any agent can call any other. Best for open-ended creative or research problems.

```
Critic Agent ⟷ Writer Agent
     ↕                ↕
Researcher ⟷ Editor Agent
```

📌 *Use for: Debate simulation, creative writing, R&D*

Multi-Agent in Practice

Automated competitive analysis report — a real business workflow

🏢 *Real-world use: A consulting firm uses a multi-agent system to auto-generate competitor analysis reports that used to take analysts 3 days. Now done in 15 minutes.*

Supervisor

Reads goal, assigns tasks, assembles final report

assigns ↙ *↓ assigns* *assigns ↘*

Web Scraper Agent

Finds competitor news, press releases, pricing

Financial Agent

Pulls revenue, growth, funding data from APIs

Writer Agent

Drafts sections of the report from raw data

results → Quality Check ↗

Quality Agent

Fact-checks, flags gaps, requests re-research

MODULE 04

LLM Observability

Seeing what's really happening inside your AI app

Why Observability Matters

If you can't see it, you can't fix it

💡 *Observability = tracing every LLM call, tool use, and decision so you can see exactly why your app gave a bad answer — and fix it fast.*

Without Observability

😤 "Why did the chatbot give the customer wrong info about our return policy?" — No way to know.

🐛 "Our agent worked yesterday, crashes today." — No logs, no idea which step failed.

💸 "Our API bill jumped 3x." — No idea which query is burning tokens.

📉 "Users hate the new feature." — No idea if it's a prompt issue or retrieval issue.

With Observability (LangSmith)

🔍 Click the failed trace → See exact prompt sent, context retrieved, answer generated. Fix in 5 min.

📊 See each node in your agent graph, its input, output, latency, and cost in a timeline.

💰 Token cost per trace, per user, per feature — find the expensive queries instantly.

📈 Compare prompt versions A/B — which one has better user thumbs-up rate?

LangSmith in Practice

The go-to observability platform for LangChain apps

🔍 Tracing

Every LLM call, tool call, retrieval step logged as a tree. Click any node to see full input/output.

Real-world use:

"Why did my RAG return the wrong document?" → See exactly what was retrieved.

📊 Datasets & Evals

Collect real user inputs, label good/bad outputs, run evals automatically on new prompt versions.

Real-world use:

"Is my new prompt better?" → Run against 100 real examples, see score improve.

🎮 Playground

Load any traced run into the playground, tweak the prompt, and re-run instantly without changing code.

Real-world use:

"I think the system prompt is wrong." → Fix and test in 2 minutes, no deploy needed.

KNOWLEDGE CHECK

Quick Quiz — Advanced P1

1. What's the difference between a regular LLM and an LLM Agent? Give a real example.
2. Describe a business workflow you'd build with LangGraph and why a linear chain wouldn't work.
3. When would you use a Supervisor + Workers multi-agent pattern instead of a pipeline?
4. Your RAG chatbot is giving wrong answers. How would you use LangSmith to debug it?

Part 1 Complete!

Modules covered in Advanced P1

✓ **01**
LLM Agents

✓ **02**
LangGraph Workflows

✓ **03**
Multi-Agent Systems

✓ **04**
Observability

Coming Up in Advanced P2 🚀

05 Guardrails

Keep your LLM safe, on-topic, and compliant

06 LLM Evaluation

Measure quality, accuracy & hallucination rate

07 Production Deployment

Scale, cost, latency — going live with LLMs

©2025 Jade Global. All rights reserved.

JADE™ GLOBAL

LLM Advanced- P2

Guardrails,
LLM Evaluation,
Production Deployment

©2025 Jade Global All rights reserved

What We'll Cover Today

Your roadmap for LLM Advanced – Part 2

05 Guardrails

Keep your LLM safe and on-topic

- Input/output validation
- Topic filtering
- PII detection
- NVIDIA NeMo Guardrails

06 LLM Evaluation

Measure quality before it reaches users

- Human vs automated eval
- RAGAs framework
- Hallucination detection
- A/B testing prompts

07 Production Deployment

Going live — cost, speed and scale

- Serving architectures
- Latency & cost tuning
- Caching strategies
- Monitoring in prod

MODULE 05

LLM Guardrails

Keeping your AI safe, on-topic, and compliant

Why Guardrails? Real AI Failures

What happens when you ship without them

💡 *Guardrails = rules and checks that sit around your LLM to control what goes in and what comes out. Think of them as a security layer for your AI.*

✈️ Air Canada

What happened:

Chatbot promised a discount that Air Canada's policy didn't offer.

Result: Customer sued. Court ruled Air Canada responsible. Paid damages.

✅ *Guardrail needed: restrict chatbot to verified policy content only (RAG + output filter).*

📦 DPD Delivery

What happened:

Customer jailbroke chatbot, made it write poems criticising DPD.

Result: Went viral on Twitter. PR disaster.

✅ *Guardrail needed: topic filter to block off-topic requests + jailbreak detection.*

💻 Samsung

What happened:

Engineers pasted confidential source code into ChatGPT for debugging.

Result: Leaked proprietary IP. Samsung banned all internal LLM use.

✅ *Guardrail needed: PII/code scanner on all outbound prompts before they reach the API.*

Types of Guardrails

Input filters, output filters, and everything in between

INPUT GUARDRAILS (before the LLM sees it)

🔓 **Jailbreak Detection**

Block prompt injection — "ignore all previous instructions"

🔏 **PII Scrubbing**

Remove emails, phone numbers, Aadhaar/SSN before sending to API

🚫 **Topic Filtering**

Block off-topic requests — politics, competitors, adult content

📏 **Token Limit**

Prevent excessively long prompts that waste API budget

OUTPUT GUARDRAILS (before the user sees it)

🎭 **Hallucination Check**

Verify answer is grounded in retrieved context (RAG faithfulness)

☠️ **Toxicity Filter**

Detect and remove harmful, offensive, or discriminatory language

✅ **Factual Accuracy**

Cross-check claims against a trusted knowledge base

📋 **Format Validation**

Ensure output is valid JSON / correct schema before returning

Implementing Guardrails

NeMo Guardrails + custom Python validators

NVIDIA NeMo Guardrails

Define rules in plain English using Colang — NeMo enforces them automatically.

```
colang rule (plain English config):


define user ask off topic
  "Tell me a joke"
  "What's your opinion on politics?"

define bot refuse off topic
  "I'm only able to help with
   questions about our products."

define flow
  user ask off topic
  bot refuse off topic
```

Custom Python Validator

Wrap your LLM call with input/output validation functions.

```
Python guard wrapper:

import re

def scrub_pii(text: str) -> str:
    # Remove email addresses
    text = re.sub(r'[w.-]+@[w.-]+', '[EMAIL]', text)
    # Remove phone numbers
    text = re.sub(r'[0-9]{10}', '[PHONE]', text)
    return text

def safe_llm_call(user_input: str) -> str:
    clean_input = scrub_pii(user_input)
    response = llm.invoke(clean_input)
    if "competitor" in response.lower():
        return "I can't comment on other products."
    return response
```

MODULE 06

LLM Evaluation

Measuring quality before your users find the problems

Why LLM Evaluation is Hard

You can't just check if the answer is 'correct'

💡 *Traditional software: input A → always output B. LLMs: input A → output varies every time. You need new ways to measure quality at scale.*

🎯 No Single Right Answer

"Summarise this article" has 1,000 valid answers. How do you know if the LLM's is good?

✅ *Fix: Use rubrics: rate on clarity, accuracy, completeness on a 1-5 scale.*

👻 Hallucination is Silent

LLMs confidently state wrong facts. "Paris is in Germany" gets the same confident tone as a correct answer.

✅ *Fix: Faithfulness check: Does the answer only use facts from the retrieved context?*

📊 Hard to Scale Manual Review

You can hire humans to rate 100 responses. But you get 10,000 responses per day in production.

✅ *Fix: LLM-as-judge: Use GPT-4 to rate other model outputs at scale automatically.*

📉 Prompt Regressions

You improve the prompt for one use case — without noticing it broke 5 other cases.

✅ *Fix: Regression test suite: run new prompts against a curated golden dataset before deploying.*

Evaluation Approaches

Three ways to measure LLM quality — and when to use each

👤 Human Evaluation

Real people rate outputs on a rubric (1–5 scale for relevance, accuracy, tone).

✅ Most accurate. Catches nuance, tone, cultural context that models miss.

⚠️ Slow (days/weeks), expensive, doesn't scale beyond hundreds of samples.

Use when: Launch evaluation, quarterly audits, high-stakes features (medical, legal).

e.g. Airbnb uses human reviewers to rate their listing-description LLM before each major release.

📐 Automated Metrics

Compute scores like BLEU, ROUGE, BERTScore against a reference answer.

✅ Instant, cheap, fully reproducible. Run in CI/CD pipeline on every code push.

⚠️ Misses meaning — a valid answer with different words scores poorly.

Use when: Translation, summarisation with known reference outputs, regression testing.

e.g. Google uses BLEU score in their Neural Machine Translation pipeline as a fast regression gate.

⚖️ LLM-as-Judge

Ask GPT-4 or Claude to rate another model's output using a detailed rubric prompt.

✅ Scales to thousands of samples. Surprisingly close to human ratings (>80% agreement).

⚠️ Biased toward same-model outputs. Costs API money. Not 100% reliable.

Use when: Daily monitoring, A/B prompt testing, RAG pipeline quality checks.

e.g. Anthropic uses Claude to evaluate Claude outputs in their constitutional AI training loop.

Evaluating RAG with RAGAs

The framework purpose-built to score your RAG pipeline

🏢 *Real use: An e-commerce company's product Q&A chatbot scored 0.55 on faithfulness. RAGAs revealed it was ignoring retrieved specs and hallucinating features. Fixed by tightening the prompt.*

Metric	What it Measures	Good Score	Low Score Means
Context Precision	Of docs retrieved, how many are actually relevant?	> 0.80	Retrieval returning noise
Context Recall	Were all the needed docs actually retrieved?	> 0.75	Missing key documents
Faithfulness	Does the answer only use facts from the context?	> 0.90	LLM is hallucinating
Answer Relevance	Does the answer address what was actually asked?	> 0.85	Answer is off-topic

A/B Testing Prompts

How to prove your new prompt is actually better

Prompt Version A (current)

```
"Answer the user's question about
 our product based on the context."
```

Prompt Version B (new)

```
"You are a helpful product expert.
 Answer concisely in 2-3 sentences.
 Only use the provided context.
 If unsure, say 'I don't know'."
```

Metric	Version A	Version B	Winner
Faithfulness	0.68	0.91	✅ B
Answer Relevance	0.74	0.88	✅ B
Avg Response Length	312 tokens	87 tokens	✅ B
User Thumbs Up	52%	78%	✅ B

MODULE 07

Production Deployment

Going live — speed, cost, and scale in the real world

The Production Gap

Why working in a notebook ≠ working in production

💡 *90% of LLM projects fail to reach production. The main reasons: latency too high, cost too high, reliability too low. This module teaches you how to fix all three.*

⚡ Latency

Dev: 5 seconds is fine in a Jupyter notebook

Prod: Users abandon after 1.5 seconds. Your chatbot needs < 800ms.

Fix: Streaming responses, smaller models, caching

💰 Cost

Dev: Calling GPT-4 for every test query costs pennies

Prod: 1M queries/day × $0.03 = $30,000/day API bill.

Fix: Model routing, semantic caching, prompt compression

🔧 Reliability

Dev: Works every time in your environment

Prod: API timeouts, rate limits, prompt injection attacks in the wild.

Fix: Retry logic, fallback models, circuit breakers

📈 Scale

Dev: 1 user at a time in dev

Prod: 1,000 concurrent users — each needing a fresh LLM call.

Fix: Async execution, load balancing, horizontal scaling

Model Selection & Routing

Not every query needs GPT-4 — smart routing saves 80% cost

💡 *Real insight from Uber: By routing simple queries to smaller models and complex ones to GPT-4, they cut LLM API costs by 70% with no drop in user satisfaction.*

Tier 1 — Small

`GPT-4o-mini, Gemini Flash, Haiku`

Cost: ~$0.001/1K tokens Latency: < 200ms

Use for: FAQ answers, simple classification, yes/no questions, format conversion

"Is this email spam?" → Haiku answers in 150ms for $0.0001

Tier 2 — Mid

`GPT-4o, Claude Sonnet, Gemini Pro`

Cost: ~$0.01/1K tokens Latency: < 800ms

Use for: RAG answers, code generation, summarisation, complex reasoning

"Summarise this 10-page report" → GPT-4o in 600ms

Tier 3 — Large

`Claude Opus, GPT-4 Turbo, Gemini Ultra`

Cost: ~$0.05/1K tokens Latency: 1–3 seconds

Use for: Legal review, medical decisions, complex multi-step reasoning, code auditing

"Review this contract for liability clauses" → Opus for $0.15

Semantic Caching & Streaming

Two techniques that massively improve perceived performance

⚡ Semantic Caching

Cache LLM responses by meaning, not exact text. Similar questions get the same cached answer instantly — no API call needed.

"What are your business hours?" → API call → cache
"When are you open?" → 0ms response from cache ✅ saves $0.02

GPTCache + LangChain:

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")
cache = {}

def ask(q):
    if q in cache:
        return cache[q]
    res = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": q}]
    )
    ans = res.choices[0].message.content
    cache[q] = ans
    return ans


print(ask("What are your business hours?"))
print(ask("When are you open?"))
```

📡 Streaming Responses

Don't wait for the full response — stream tokens to the user as they generate. First token appears in < 200ms instead of waiting 3 seconds.

Without streaming: user stares at blank screen for 4 seconds
With streaming: first word appears in 180ms ✅ feels 10x faster

LangChain streaming to frontend:

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")

stream = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Explain RAG in
3 points"}],
    stream=True
)

for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="",
flush=True)
```

Deployment Architectures

Three proven patterns for taking LLMs to production

☁️ Serverless (Managed API)

Call OpenAI/Anthropic/Gemini directly from your app. No infra to manage.

✅ Zero setup, instant scale, always up-to-date models

⚠️ Data leaves your infra, ongoing API cost, vendor lock-in

📌 *Best for: Startups, MVPs, low-to-medium traffic apps*

e.g. ChatPDF, most LLM startups in 2023–24

🖥️ Self-Hosted OSS Model

Run open-source models (Llama 3, Mistral) on your own GPU infra via Ollama or vLLM.

✅ Data stays private, predictable cost at scale, full control

⚠️ High upfront GPU cost, ops burden, smaller models than GPT-4

📌 *Best for: Healthcare, finance, legal — any regulated industry*

e.g. Hospitals, banks, government agencies using Llama 3

⚖️ Hybrid Routing

Simple/sensitive queries → local model. Complex queries → cloud API. Best of both worlds.

✅ Optimises cost + privacy + quality simultaneously

⚠️ Complex routing logic to build and maintain

📌 *Best for: Enterprises with mixed workloads and compliance requirements*

e.g. Shopify routes product FAQ to local, complex returns to GPT-4

Production Monitoring

What to track once your LLM app is live

🏢 *"We had no idea our chatbot was costing us ₹4 lakh a day until we set up monitoring." — A real fintech team story. Track everything from day one.*

⚡ Performance

- P50/P95/P99 latency per endpoint
- Token generation speed (tok/sec)
- Time to first token (TTFT)
- Cache hit rate

💰 Cost

- Total API spend per day/week
- Cost per user session
- Cost per query type
- Token usage by model tier

✅ Quality

- User thumbs up/down rate
- Hallucination flag rate
- Topic filter trigger rate
- Failed/empty responses

🚨 Errors

- API error rate (429, 500s)
- Timeout rate
- Guardrail violations per hour
- Retry success/fail rate

Cost Optimisation Playbook

Practical steps to reduce your LLM bill without hurting quality

#	Tactic	Cost Saving	How to Implement
1	**Prompt Compression**	**20–40%**	Remove filler words, collapse repetition, use shorter system prompts. Tool: LLMLingua.
2	**Semantic Caching**	**30–60%**	Cache responses for similar queries. Hit rate of 40% is common on FAQ chatbots. Tool: GPTCache.
3	**Model Routing**	**40–70%**	Route simple queries to cheap models (Haiku/Mini), only hard ones to GPT-4. Tool: LiteLLM Router.
4	**Batch Processing**	**50%**	Non-urgent tasks (email drafts, report gen) sent in batch API at half-price. Tool: OpenAI Batch API.
5	**Context Window Management**	**15–30%**	Summarise old chat history instead of sending full history each time. Keep only last 5 turns + summary.

Production Launch Checklist

Before you go live — tick every box

🛡️ Safety & Guardrails

- ☐ Input PII scrubbing enabled
- ☐ Jailbreak / prompt injection detection
- ☐ Output topic filter tested
- ☐ Fallback message for filtered responses

📊 Evaluation

- ☐ Golden eval dataset (100+ examples) created
- ☐ Faithfulness score > 0.85 on RAG pipeline
- ☐ A/B test shows new prompt beats baseline
- ☐ Human review of 50 edge cases passed

⚡ Performance

- ☐ P95 latency < 2 seconds under load test
- ☐ Streaming enabled for all user-facing calls
- ☐ Semantic cache configured and tested
- ☐ Retry + exponential backoff implemented

💰 Cost & Monitoring

- ☐ Model routing rules configured
- ☐ Daily spend alert set (e.g. > ₹50K)
- ☐ LangSmith tracing enabled
- ☐ Error rate dashboard + on-call alert live

KNOWLEDGE CHECK

Quick Quiz — Advanced P2

1. Name 2 real-world examples of LLM failures that guardrails would have prevented.

2. Your RAG chatbot scores 0.58 on faithfulness. What does that mean and how do you fix it?

3. A user sends 50,000 queries/day. How would you reduce API cost without hurting quality?

4. What 3 metrics would you monitor on day 1 of going live, and why?

Advanced Level Complete! 🏆

All 7 Advanced Modules Covered

✓ 01
LLM Agents

✓ 02
LangGraph Workflows

✓ 03
Multi-Agent Systems

✓ 04
Observability

✓ 05
Guardrails

✓ 06
LLM Evaluation

✓ 07
Production Deployment

🎓 **You are now an LLM Engineer. Build something great.**

🔬 **Fine-tuning**
Train models on your data

🌐 **LLM Research**
Papers: RLHF, Constitutional AI

💼 **Build Products**
Ship your first LLM app

🤝 **Community**
LangChain Discord, HuggingFace

©2025 Jade Global. All rights reserved.